

FULLY NEURAL NETWORK BASED MODEL FOR GENERAL TEMPORAL POINT PROCESSES



- Generalized the time evolution of the conditional intensity function
- The researchers formulated the intensity function based on a neural network rather than assuming a specific functional form.

OTHER STUDIES

- used numerical approximations to evaluate the integral, numerical approximations:
 - deteriorates fitting accuracy
 - computationally expensive

OVERCOMING LIMITATIONS

- Modeled the integral of the conditional intensity function using a feedforward neural network.
 - The conditional intensity function is obtained by differentiating it.
- This approach enabled them to exactly evaluate the log-likelihood function of the general model without numerical approximations.
- Finally, they show the effectiveness of our proposed model by analyzing synthetic and real datasets.

METHODS 2.1

A temporal point process is a stochastic process that generates a sequence of discrete events at times $\{t_i\}_{i=1}^n$ in a given observation interval $[0, T]$. The process is characterized via a conditional intensity function $\lambda(t|H_t)$, which is the intensity function of the event at the time t conditioned on the event history $H_t = \{t_i | t_i < t\}$ up to the time t , given as follows:

$$\lambda(t|H_t) = \lim_{\Delta \rightarrow 0} \frac{P(\text{one event occurs in } [t, t + \Delta) | H_t)}{\Delta}. \quad (1)$$

If the conditional intensity function is specified, the probability density function of the time t_{i+1} of the next event, given the times $\{t_1, t_2, \dots, t_i\}$ of the past events, is obtained as follows:

$$p(t_{i+1} | t_1, t_2, \dots, t_i) = \lambda(t_{i+1} | H_{t_{i+1}}) \exp \left\{ - \int_{t_i}^{t_{i+1}} \lambda(t | H_t) dt \right\}, \quad (2)$$

where the exponential term in the right-hand side represents the probability that no events occur in $[t_i, t_{i+1})$. The probability density function to observe an event sequence $\{t_i\}_{i=1}^n$ is then obtained as follows:

$$p(\{t_i\}_{i=1}^n) = \prod_{i=1}^n \lambda(t_i | H_{t_i}) \exp \left\{ - \int_0^T \lambda(t | H_t) dt \right\}. \quad (3)$$

The most basic example of a temporal point process is a stationary Poisson process, which assumes that the events are independent of each other. The conditional intensity function of the stationary Poisson process is given as $\lambda(t|H_t) = \lambda$. Another popular example is the Hawkes process [1, 2, 3], which is a simple model of a self-exciting point process. The conditional intensity function of the Hawkes process is given as $\lambda(t|H_t) = \mu + \sum_{t_i < t} g(t - t_i)$, where $g(s)$ is a kernel function ($g(s) = 0$ if $s < 0$) that represents the triggering effect from the past event.

METHODS 2.2

A hidden state h_i of the RNN is updated as follows:

$$h_i = f(W^h h_{i-1} + W^x x_i + b^h)$$

- where W^h , W^x , and b^h denote the recurrent weight matrix, input weight matrix, and bias term, respectively, and f is an activation function.

The conditional intensity function is then formulated as a function of the elapsed time from the most recent event and the hidden state of the RNN, given as follows:

$$\lambda(t|H_t) = \phi(t - t_i | h_i)$$

- where ϕ is a non-negative function referred to as a hazard function.

METHODS 2.2

Du et al. (2016) assumed the following form for the hazard function:

$$\phi(\tau|h_i) = \exp(w^t \tau + v^\phi \cdot h_i + b^\phi)$$

- The exponential function in the above equation is used to ensure the non-negativity of the intensity.

A simplified model, where the conditional intensity function is constant over the period between the successive events, was also considered:

$$\phi(\tau|h_i) = \exp(v^\phi \cdot h_i + b^\phi)$$

- In this model, the inter-event interval $\tau_i = t_{i+1} - t_i$ follows the exponential distribution with mean $1 / \exp(v^\phi \cdot h_i + b^\phi)$.

METHODS 2.2

The log-likelihood function of the RNN based model can be obtained:

$$\log L(\{t_i\}) = \sum_i \left[\log \phi(t_{i+1} - t_i | \mathbf{h}_i) - \int_0^{t_{i+1} - t_i} \phi(\tau | \mathbf{h}_i) d\tau \right]$$

- The parameter values of the model are estimated by maximizing the log-likelihood function. For this purpose, the backpropagation through time (BPTT) is employed to obtain the gradient of the log-likelihood function.

Only the dependence on a fixed number d of the most recent events is considered, where d is a hyper-parameter called the truncation depth.

Namely, for each time index i , the hidden state $\mathbf{h}_i$ is obtained by feeding the inputs $\{\mathbf{x}_j\}_{j=i-d+1}^i$ from the d most recent events to the RNN.

METHODS 2.4

Rather than directly modeling the hazard function, we herein propose to model the cumulative hazard function $\Phi(\tau|h_i)$, defined as follows:

$$\Phi(\tau|h_i) = \int_0^\tau \phi(s|h_i)ds$$

The hazard function itself can be then obtained by differentiating the cumulative hazard function with respect to τ as follows:

$$\phi(\tau|h_i) = \frac{\partial}{\partial \tau} \Phi(\tau|h_i)$$

The log-likelihood function is reformulated as follows using the cumulative hazard function:

$$\log L(\{t_i\}) = \sum_i \left[\log \left\{ \frac{\partial}{\partial \tau} \Phi(\tau = t_{i+1} - t_i | h_i) \right\} - \Phi(t_{i+1} - t_i | h_i) \right]$$

METHODS 2.4

— positive weight

The activation functions of the hidden units and the output unit are set to be the tanh function and the softplus function, $\log(1 + \exp(\cdot))$

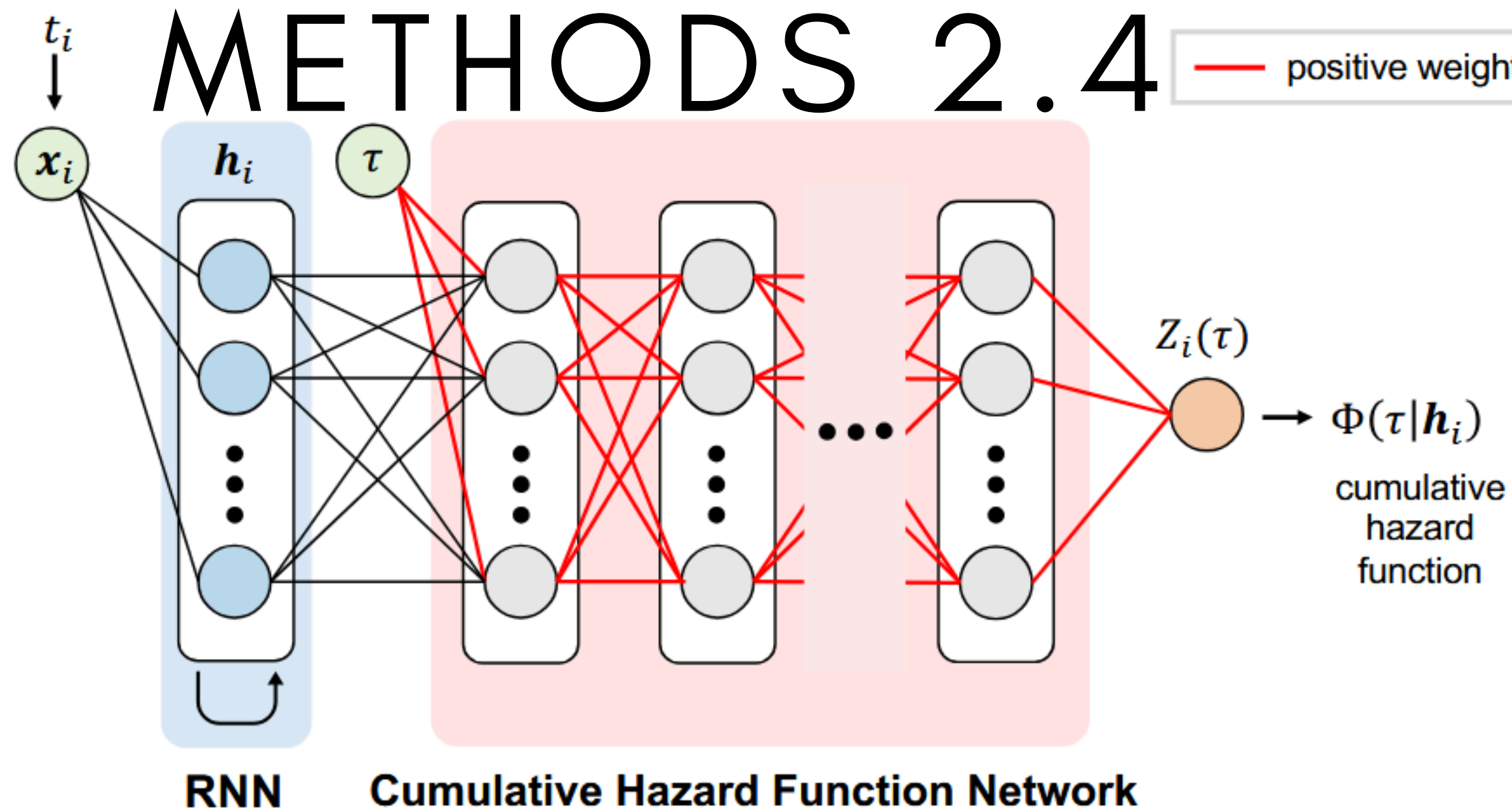


Figure 1: Network architecture to output the cumulative hazard function.

- The cumulative hazard function is modeled using a feedforward neural network for flexible modeling.
- The cumulative hazard function is a monotonically increasing function of τ and is positive-valued.

METHODS 2.4

The cumulative hazard function $\Phi(\tau|\mathbf{h}_i)$ and the hazard function $\phi(\tau|\mathbf{h}_i)$ are now formulated as follows based on the output $Z_i(\tau)$ of the cumulative hazard function network:

$$\Phi(\tau|\mathbf{h}_i) = Z_i(\tau), \tag{12}$$

$$\phi(\tau|\mathbf{h}_i) = \frac{\partial}{\partial \tau} \Phi(\tau|\mathbf{h}_i) = \frac{\partial}{\partial \tau} Z_i(\tau). \tag{13}$$

EXPERIMENTS 4.0

- The number of units in the RNN is fixed to 64 for all the models.
- The first model is equipped with the constant hazard function (**the constant model**)
- The exponential hazard function was used in the second model (**the exponential model**)
- the parameters are almost the same between the third and fourth models.

EXPERIMENTS 4.0

- The number of units in the RNN is fixed to 64 for all the models.
- The third model employs the piecewise constant hazard function (**the piecewise constant model**).
- The researchers set $\tau_{\max}$ to the maximum value of the inter-event interval in each dataset and use the condition $l = \tau_{\max}/128$.
- The fourth model employs the neural network based hazard function proposed in this study (**the neural network based model**).
- For this model, two hidden layers were used for the cumulative hazard function network, and the number of units in each layer is 64. In this setting, the numbers of the parameters are almost the same between the third and fourth models.

EXPERIMENTS 4.0

- Each dataset is divided into the training and test data.
- In the training phase, the model parameters are estimated using the training data:
 - For this optimization, the Adam optimizer with the learning rate 0.001, $\beta_1 = 0.9$, and $\beta_2 = 0.999$ is used, and the batch size is 256.
 - The truncation depth d from [5, 10, 20, 40] using 20% of the training data.

EXPERIMENTS 4.0

- In the test phase, the predictive performances of the trained models is evaluated using the test data:
 - In each time step, the probability density function $p^*(t|t_1, t_2, \dots, t_i)$ of the time of the coming event given the past events $\{t_1, t_2, \dots, t_i\}$ is calculated from eq. (2):
$$p(t_{i+1}|t_1, t_2, \dots, t_i) = \lambda(t_{i+1}|H_{t_{i+1}}) \exp \left\{ - \int_{t_i}^{t_{i+1}} \lambda(t|H_t) dt \right\}, \quad (2)$$
 - and scored by the negative log-likelihood - $\log p^*(t_{i+1}|t_1, t_2, \dots, t_i)$ for the actually observed t_{i+1} (a smaller score means a better predictive performance)
- The score is finally averaged over the events in the test data.
- Computations were performed under a GPU environment provided by Google Colaboratory.

EXPERIMENTS 4.1

We use the synthetic data generated from the following stochastic processes. In this experiment, 100,000 events are generated from each process, and 80,000/20,000 events are used for training/testing.

Stationary Poisson Process (S-Poisson): The conditional intensity function is given as $\lambda(t|H_t) = 1$.

Non-stationary Poisson Process (N-Poisson): The conditional intensity function is given as $\lambda(t|H_t) = 0.99 \sin(2\pi t/20000) + 1$.

Stationary Renewal Process (S-Renewal): In this process, the inter-event intervals $\{\tau_i = t_{i+1} - t_i\}$ are independent and identically distributed according to a given probability density function $p(\tau)$. We herein use the log-normal distribution with a mean of 1.0 and a standard deviation of 6.0 for $p(\tau)$. In this setting, a generated sequence exhibits a bursty behavior: multiple events tend to occur in a short period and are followed by a long silence period like the burst firing of biological neurons.

Non-stationary Renewal Process (N-Renewal): A sequence $\{t_i\}$ following a non-stationary renewal process is obtained as follows [17]: we first generate a sequence $\{t'_i\}$ from a stationary renewal process, and then we rescale the time according to $t'_i = \int_0^{t_i} r(t)dt$ for a non-negative trend function $r(t)$. We use the gamma distribution with a mean of 1.0 and a standard deviation of 0.5 to generate the stationary renewal process and set the trend function to $r(t) = 0.99 \sin(2\pi t/20000) + 1$. In this process, an inter-event interval tends to be followed by the one with similar length, but the expected length gradually varies in time.

Self-correcting Process (SC): The conditional intensity function is given as $\lambda(t|H_t) = \exp(t - \sum_{t_i < t} 1)$.

Hawkes Processes (Hawkes1 and Hawkes2): We use the Hawkes process, in which the kernel function is given by the sum of multiple exponential functions: the conditional intensity function is given by $\lambda(t|H_t) = \mu + \sum_{t_i < t} \sum_{j=1}^M \alpha_j \beta_j \exp\{-\beta_j(t - t_i)\}$. For the **Hawkes1** model, we set $M = 1, \mu = 0.2, \alpha_1 = 0.8$, and $\beta_1 = 1.0$. For the **Hawkes2** model, we set $M = 2, \mu = 0.2, \alpha_1 = 0.4, \beta_1 = 1.0, \alpha_2 = 0.4$, and $\beta_2 = 20.0$. Compared to the Hawkes1 model, the kernel function of the Hawkes2 model rapidly varies in time for small τ .

EXPERIMENTS 4.1

predicting skill for synthetic data

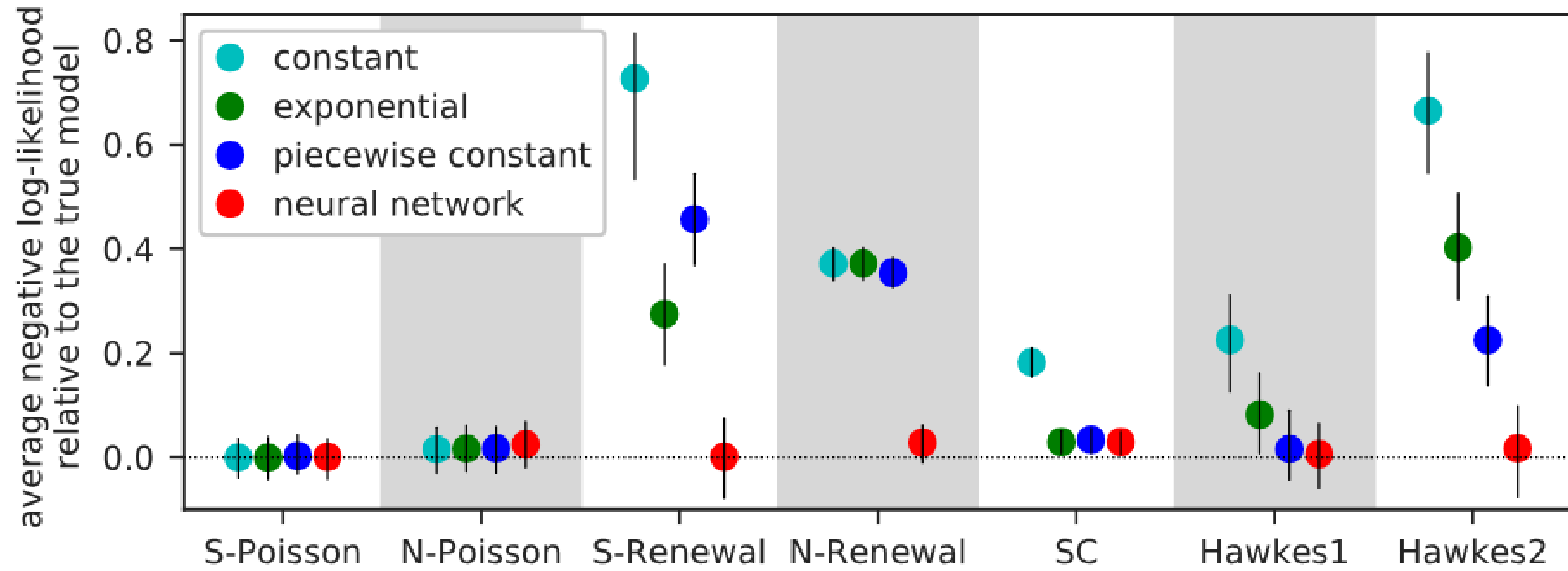


Figure 2: Performances for the synthetic datasets. For each synthetic dataset, we evaluate the predictive performances of the four RNN based models according to the negative log-likelihood averaged per event calculated for the test data (the smaller is more preferable). Each score in the panel is standardized by subtracting the score of the true model. Each error bar represents the 25% and 75% percentiles of the score calculated for every 300 samples.

EXPERIMENTS 4.1

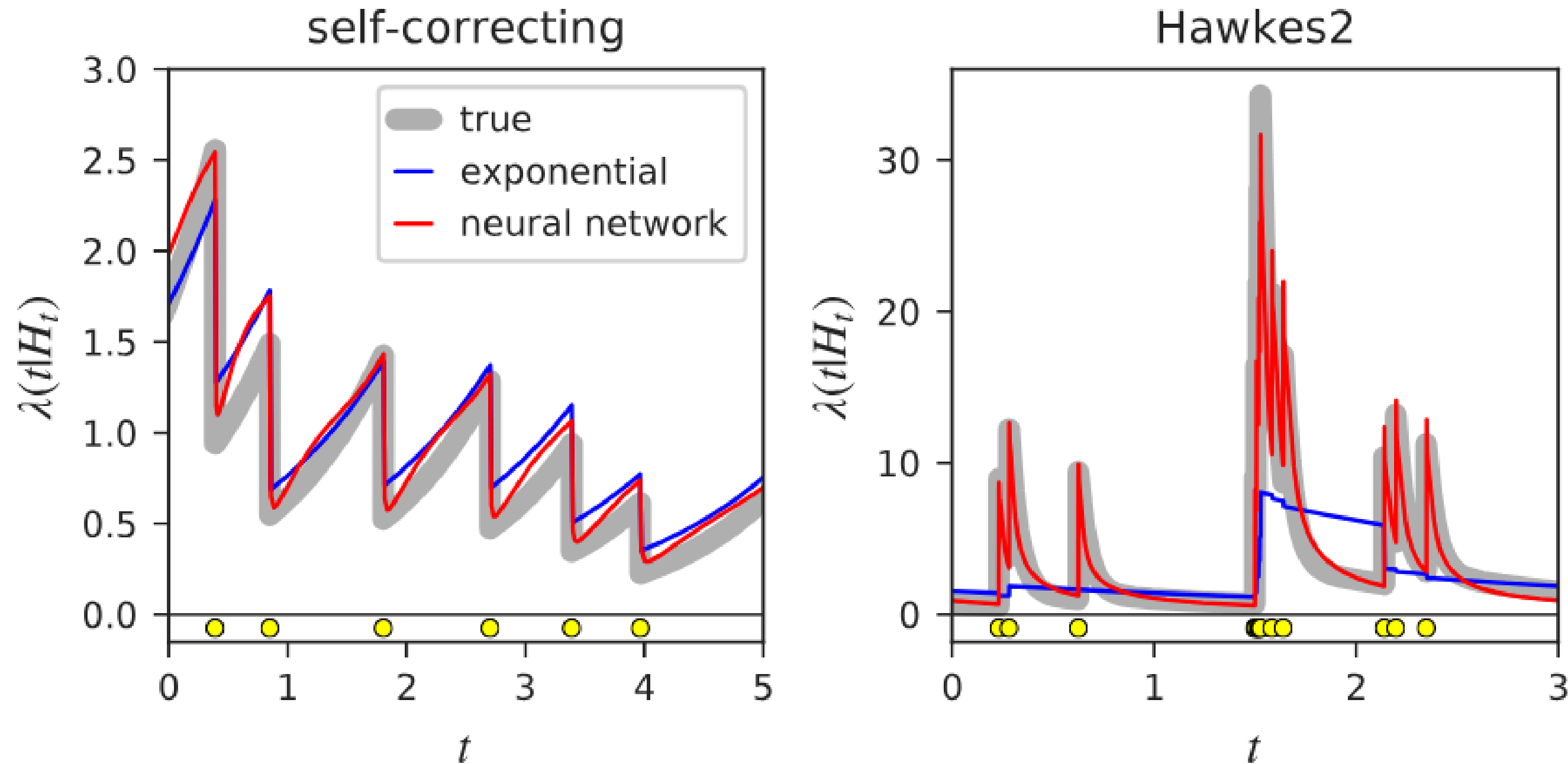


Figure 3: The conditional intensity functions estimated from the exponential model and the neural network based model are compared with the true one. The yellow circles in the bottom of each panel represent the events. The exponential hazard function is valid for the self-correcting process, but not for the Hawkes2 process.

EXPERIMENTS 4.2

4.2 Real data

We use the following real datasets for the next experiment.

Finance dataset: This dataset contains the trading records of Nikkei 225 mini, which is the most liquid features contracts in Asia [18]. The timestamps of 182,373 transactions in one day are analyzed, and the first 80% and the last 20% of the data are used for training and testing, respectively.

Emergency call dataset: This dataset contains the records of the police department calls for service in San Francisco [19]. Each record contains the timestamp and address from which the call was made. We prepare 100 separate sequences for the 100 most frequent addresses, which contain a total of 294,865 events. The first 80% and the last 20% of the events in the sequences are used for training and testing, respectively.

Meme dataset: MemeTracker tracks the popular phrases from numerous online resources such as news media and personal blogs [20]. This dataset records the timestamps when the focused phrases appear on the internet. We first extract the 50 most frequent phrases and obtain the corresponding 50 separate sequences. We use 40 sequences out of 50 with 247,579 events for training and the remaining 10 sequences with 61,095 events for testing.

Music dataset: This dataset records the history of music listening of users at <https://www.last.fm/> [21]. We prepare the 100 sequences for the 100 most active users in Jan 2009, which contain a total of 299,046 events. The first 80% and the last 20% of the events in the sequences are used for training and testing, respectively.

EXPERIMENTS 4.2

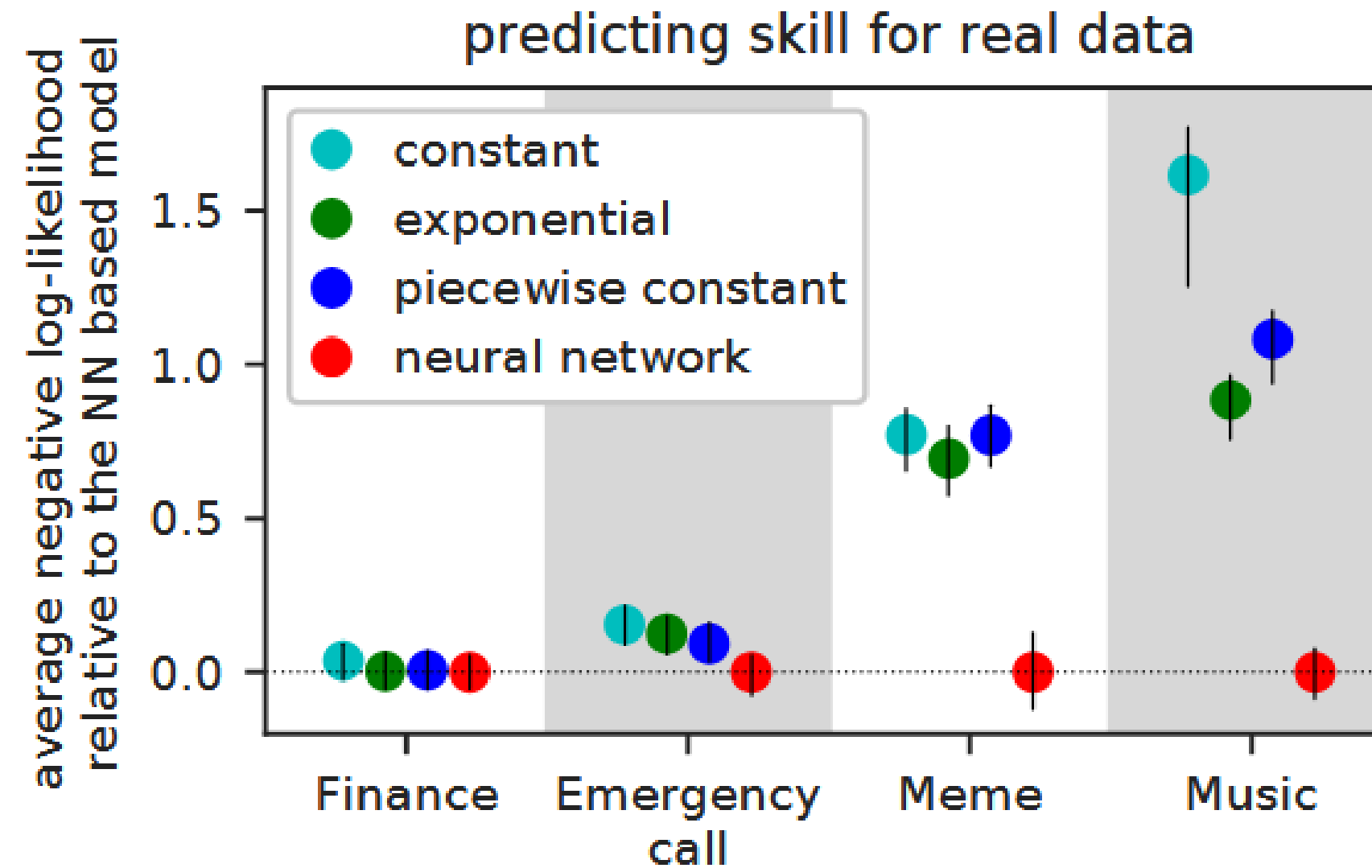


Figure 4: Performances for the real datasets. The score of each RNN based model is standardized by subtracting the score of the neural network based model.

EXPERIMENTS 4.3

- The researchers carried out the time prediction experiments.
- Specifically, they used the median of the predictive distribution to predict the timing of the coming event and evaluate the prediction by the mean absolute error.

	S-Poisson	N-Poisson	S-Renewal	N-Renewal	SC	Hakes1	Hawkes2	Finance	Call	Meme	Music	AVERAGE
constant	0.696	0.704	1.085	0.450	0.543	0.915	1.075	0.883	0.876	0.848	1.066	0.831
exponential	0.696	0.716	0.917	0.452	0.498	0.854	0.986	0.840	0.861	0.823	0.788	0.766
piecewise const.	0.696	0.716	0.982	0.437	0.494	0.850	0.965	0.839	0.862	0.826	0.856	0.775
neural network	0.696	0.710	0.894	0.414	0.496	0.848	0.962	0.847	0.853	0.811	0.783	0.756

- In the table, the best score is in bold, and it is in red if the difference between the best score and the second best score is statistically significant ($p < 0.01$).

EXPERIMENTS 4.4

- Both the two models aim at flexibly estimating the intensity function. The technical advantage of NN model over the CT-LSTM model is that the log-likelihood function can be exactly evaluated,
 - therefore the estimation can be carried out efficiently
 - For the CT-LSTM model, the evaluation of the log-likelihood function is based on the Monte Carlo method,
 - this can be computationally expensive and can deteriorate the performance.

EXPERIMENTS 4.4

- the predictive performance of the CT-LSTM model in terms of the mean negative log-likelihood (MNLL) and the mean absolute error (MAE).
- The number of the hidden units in the CT-LSTM model is set to 42, so that the numbers of the free parameters are almost the same between the CT-LSTM model and the NN model.
- The following table lists the scores of the CT-LSTM model
 - relative to the NN model; the positive score means that the NN model is better than the CT-LSTM model. In the table, the score is in blue if the NN model is significantly better than the CT-LSTM model ($p < 0.01$).

	S-Poisson	N-Poisson	S-Renewal	N-Renewal	SC	Hakes1	hawkes2	Finance	Call	Meme	Music	AVERAGE
MNLL	-0.002	-0.013	0.012	-0.007	-0.008	-0.002	0.002	-0.011	-0.017	0.590	0.163	0.064
MAE	0.000	-0.007	0.000	0.042	0.016	0.002	0.005	-0.013	-0.002	-0.002	0.026	0.009

DISCUSSION AND CONCLUSIONS

- By using a feed-forward neural network, the NN model can reproduce any time course of the hazard function in principle, i.e., the usefulness of fully neural network based modeling for point processes is indicated.
- By modeling the cumulative hazard function rather than the hazard function itself, we can avoid the direct evaluation of the integral in the log-likelihood function.
- The log-likelihood function can be exactly and efficiently evaluated without relying on the numerical approximation because of this approach.